# CORE103L Computational Thinking
# Pre-LW5 - Nested Iteration Practice

Department of Computer Science
Dhanani School of Science and Engineering
Habib University
Fall 2026

**Purpose.** This practice set prepares you for LW5 by focusing on nested iteration. Each question must be implemented as a function. Do not use `input()`.

**General implementation requirements.**

- Use nested iteration in every question.
- Do not hardcode the output.
- Do not use string multiplication.
- Do not use `join()`, comprehensions, or generator expressions to generate complete pattern rows.

## Question 1: Square Pattern

This question should be implemented in `q1.py`.
Given a positive integer `size`, print a square pattern of asterisks.

```
>>> square_pattern(4)
****
****
****
****
```

### Constraints

- `isinstance(size, int)` is `True`.
- `size` $\geq 1$
- Do not use string multiplication.

**Function Specification.** Write a function named `square_pattern(size)` that prints the square pattern illustrated above. The function must print exactly `size` rows.

## Testing

To test your function, type:

```
pytest tests/test_q1.py
```

## Question 2: Rectangle Pattern

This question should be implemented in `q2.py`.
Given positive integers `height` and `width`, print a rectangle pattern of asterisks.

```
>>> rectangle_pattern(3, 5)
*****
*****
*****
```

### Constraints

- `isinstance(height, int)` and `isinstance(width, int)` are `True`.
- `height` $\geq 1$ and `width` $\geq 1$
- Do not use string multiplication.

**Function Specification.** Write a function named `rectangle_pattern(height, width)` that prints the rectangle illustrated above.

### Testing

To test your function, type:

```
pytest tests/test_q2.py
```

## Question 3: Right Triangle Pattern

This question should be implemented in `q3.py`.
Given a positive integer `height`, print the right-triangle pattern illustrated below.

```
>>> right_triangle(5)
*
**
***
****
*****
```

### Constraints

- `isinstance(height, int)` is `True`.
- `height` $\geq 1$
- Do not use string multiplication.

**Function Specification.** Write a function named `right_triangle(height)` that prints exactly `height` rows following the pattern shown above.

### Testing

To test your function, type:

```
pytest tests/test_q3.py
```

## Question 4: Half Diamond Pattern

This question should be implemented in `q4.py`.
Given a positive integer `height`, print the half-diamond pattern illustrated below.

```
>>> half_diamond(4)
*
**
***
****
***
**
*
```

### Constraints

- `isinstance(height, int)` is `True`.
- `height` $\geq 1$
- Do not use string multiplication.

**Function Specification.** Write a function named `half_diamond(height)` that prints the pattern shown above.

### Testing

To test your function, type:

```
pytest tests/test_q4.py
```

## Question 5: Hollow Square Pattern

This question should be implemented in `q5.py`.
Given a positive integer `size`, print a hollow square pattern of asterisks.

```
>>> hollow_square(5)
*****
*   *
*   *
*   *
*****
```

**Constraints**

- `isinstance(size, int)` is `True`.
- `size` $\geq 1$
- Do not use string multiplication.

**Function Specification.** Write a function named `hollow_square(size)` that prints the hollow-square pattern shown above. Spaces inside the square are part of the required output.

### Testing

To test your function, type:

```
pytest tests/test_q5.py
```

## Question 6: Chessboard Pattern

This question should be implemented in `q6.py`.
Given positive integers `rows` and `columns`, print a chessboard pattern using `X` and `O`. The top-left character must be `X`, and the arrangement must follow the sample.

```
>>> chessboard_pattern(5, 5)
XOXOX
OXOXO
XOXOX
OXOXO
XOXOX
```

**Constraints**

- `isinstance(rows, int)` and `isinstance(columns, int)` are `True`.
- `rows` $\geq 1$ and `columns` $\geq 1$
- Do not use string multiplication.

**Function Specification.** Write a function named `chessboard_pattern(rows, columns)` that prints exactly `rows` rows and `columns` characters per row, following the pattern shown above.

### Testing

To test your function, type:

```
pytest tests/test_q6.py
```